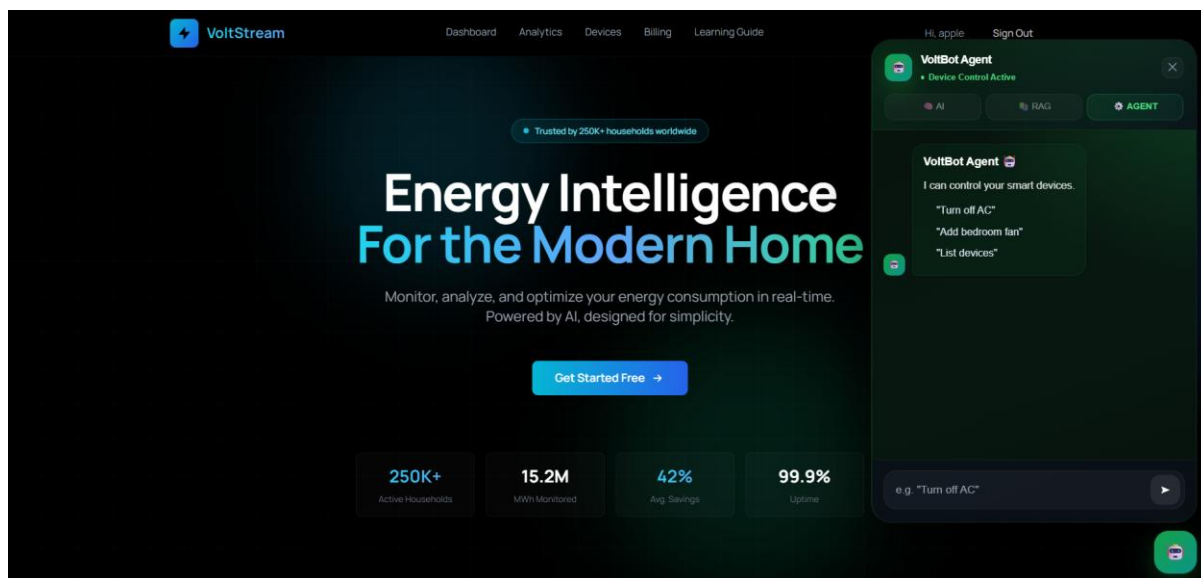


VOLTSTREAM (Week 4)

Intelligent Energy Management Platform

Single Agent Integration using ADK

Source Code: <https://github.com/MYethishwar/VoltStream>



Week 4 introduced a complete Agentic AI layer to VoltStream using Google Agent Development Kit, implementing a router agent pattern with three specialist sub-agents — device control, device manager, and energy advisor — each connected to MongoDB through six custom tool functions. A JWT-authenticated FastAPI endpoint was added to wire the agent into the existing backend, and the VoltBot ChatWidget was extended with a dedicated Agent mode, enabling users to control devices, register new devices, and receive energy saving advice entirely through natural language commands.

Project Architecture

VoltStream Agent Architecture - Workflow

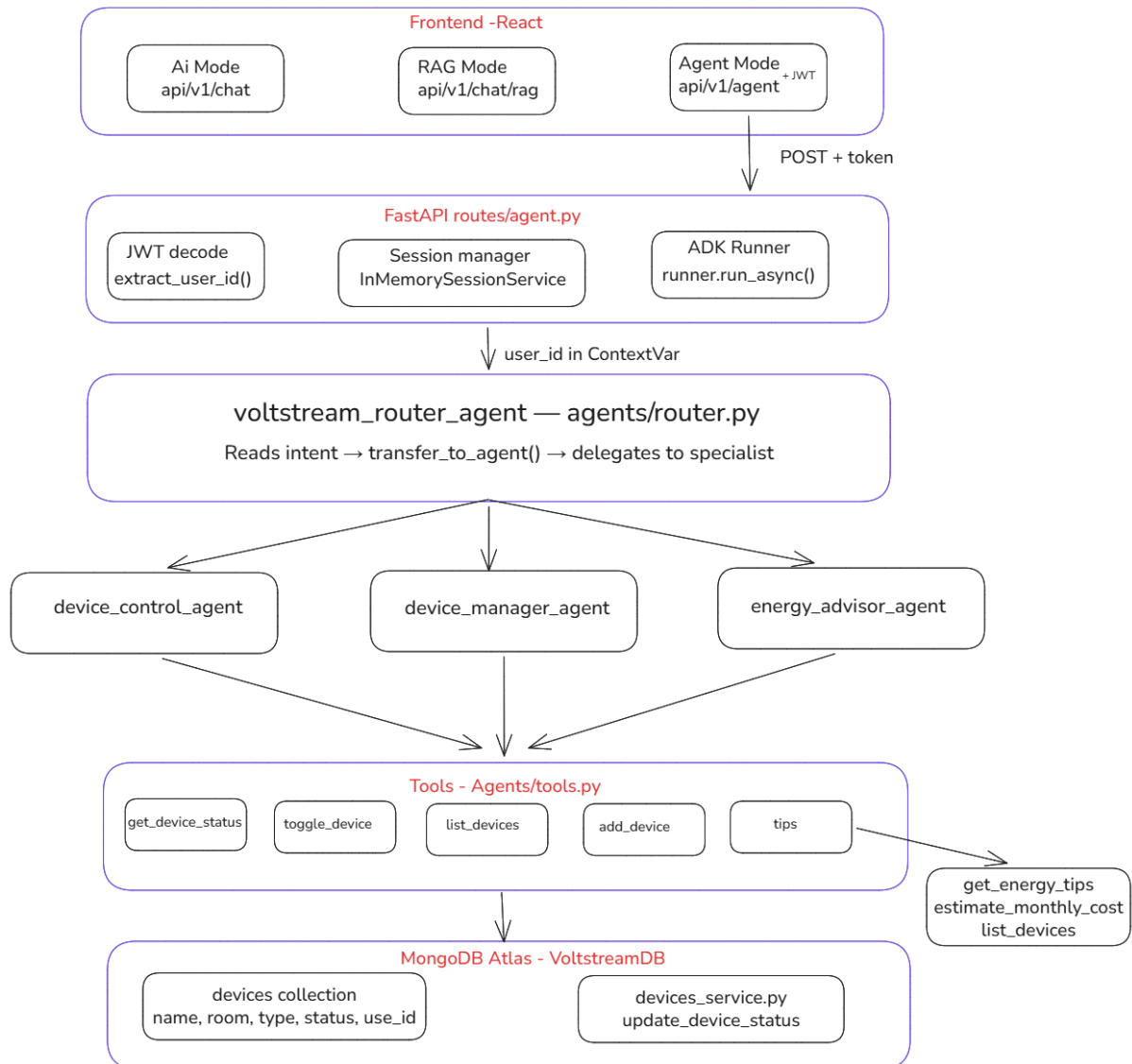


Table of Contents

1. Introduction & Objectives
2. AI Agent
3. VoltStream Agent Architecture
4. Implementation Details
5. Output & Results

1. Introduction & Objectives

Week 4 of VoltStream focused on implementing Agentic AI using Google Agent Development Kit (ADK). The goal was to build an intelligent device-control agent capable of understanding natural language commands and executing real actions on smart home devices stored in MongoDB.

Instead of manually clicking toggles on the Devices page, users can now send messages like "Turn off the Air Conditioning" and the agent automatically identifies the correct device, calls the appropriate tool, updates the database, and returns a human-readable confirmation.

Objectives

1. Understand the AI agent loop — plan, select, execute, observe, respond
 2. Set up Google ADK and configure Gemini API credentials
 3. Define Python functions as ADK tools with proper docstrings
 4. Build a VoltStream Device Control Agent with four custom tools
 5. Implement a router agent pattern with three specialist sub-agents
 6. Wire the agent into VoltStream via a JWT-authenticated FastAPI endpoint
 7. Add Agent mode to the existing VoltBot ChatWidget
 8. Test and demonstrate the full agent loop using ADK web interface
-

2. AI Agent

An AI agent is a system that uses a large language model to reason about a problem, select the appropriate tool, execute it, observe the result, and return a response. Unlike a simple chatbot that only generates text, an agent can take real actions — such as updating a database, calling an API, or controlling a device.

The Agent Loop

The agent loop is the core reasoning cycle that runs on every user message.

Step 1 — Plan: The LLM reads the user message, system instruction, and available tool descriptions. It reasons about what the user wants.

Step 2 — Select: The LLM decides which tool to call and what arguments to pass.

Step 3 — Execute: ADK calls the selected Python function with the extracted arguments. The function performs a real action such as querying or updating MongoDB.

Step 4 — Observe: The tool's return value is fed back to the LLM so it understands what happened.

Step 5 — Respond: The LLM generates a natural language reply based on the tool result.

Example — Checkpoint Demo

User message: "Turn off the Air Conditioning"

Step	What Happens
Plan	User wants to turn something off

Step	What Happens
Select	<code>toggle_device("Air Conditioning", False)</code>
Execute	MongoDB update — device status set to False
Observe	"Air Conditioning has been turned OFF successfully"
Respond	"Done! The Air Conditioning has been turned OFF."

This loop is fully visible in the ADK web interface under the Events tab as numbered events showing each step including tool calls, tool results, and the final response.

3. VoltStream Agent Architecture

Router Agent Pattern

The VoltStream agent system follows a Router Agent pattern. A root router agent reads user intent and delegates to one of three specialist sub-agents using ADK's built-in `transfer_to_agent` mechanism.

`voltstream_router_agent` (root — routes only, never answers directly)

- └─ `device_control_agent` → ON/OFF control, status check, list devices
- └─ `device_manager_agent` → add new devices
- └─ `energy_advisor_agent` → energy tips, monthly cost estimates

The router uses `gemini-2.0-flash` for fast routing decisions. Sub-agents use `gemini-2.5-flash` for accurate tool calling and response generation.

How Routing Works

1. User sends a message to `POST /api/v1/agent`
2. FastAPI decodes the JWT token and extracts `user_id`
3. ADK Runner passes the message to the router agent
4. Router reads intent and calls `transfer_to_agent()` to the correct sub-agent
5. Sub-agent selects and calls the appropriate tool
6. Tool queries or updates MongoDB scoped to the logged-in user
7. Result returns through sub-agent back to the user

The `transfer_to_agent` function is not written by the developer — it is automatically created by ADK when `sub_agents` are declared. Once a session is routed to a sub-agent, follow-up messages on the same topic go directly to that sub-agent without re-routing, which improves response speed.

Agents Folder Structure

`backend/`

`agents/`

`__init__.py` `exports root_agent = router_agent`

```

router.py      voltstream_router_agent
agent_service.py device_control_agent
device_manager.py device_manager_agent
energy_advisor.py energy_advisor_agent
tools.py       all tool functions + ContextVar
.env           GEMINI_API_KEY, GOOGLE_GENAI_USE_VERTEXAI=false

```

4. Implementation Details

All tools are defined as Python functions in `agents/tools.py`. ADK uses the function signature and docstring to understand when and how to call each tool.

Tool	Agent	Purpose
<code>get_device_status</code>	<code>device_control_agent</code>	Returns current ON/OFF state of a device
<code>toggle_device</code>	<code>device_control_agent</code>	Turns a device ON or OFF in MongoDB
<code>list_devices</code>	<code>device_control_agent</code> / <code>energy_advisor_agent</code>	Lists all user devices with status
<code>add_device</code>	<code>device_manager_agent</code>	Adds a new device to MongoDB
<code>get_energy_tips</code>	<code>energy_advisor_agent</code>	Returns saving tips for a device type
<code>estimate_monthly_cost</code>	<code>energy_advisor_agent</code>	Calculates monthly electricity cost

Smart Device Matching

The `find_device()` function uses a 3-tier fuzzy matching algorithm to handle natural language device names:

Tier 1 — Exact match: "fan" matches only a device named exactly "fan"

Tier 2 — Word set match: "living room fan" matches "Living Room Fan" by comparing word sets in both directions

Tier 3 — Substring fallback: "ac" matches "Living Room AC" as a last resort

When multiple devices match, the function picks the one whose name length is closest to the query. This prevents ambiguous matches when users have multiple similar devices.

Per-User Data Isolation

A Python `ContextVar` called `current_user_id` is defined in `tools.py` and set per-request in `routes/agent.py` from the decoded JWT token. Every tool reads this value to filter MongoDB queries to the logged-in user's devices only.

python

```
current_user_id: ContextVar[str] = ContextVar("current_user_id", default="demo_user")
```

ContextVar is async-safe and per-task, meaning concurrent requests from different users never interfere with each other. The variable is always reset in a finally block after the request completes.

API Endpoint

The agent is exposed via a single FastAPI endpoint:

POST /api/v1/agent

Header: Authorization: Bearer <token>

Body: { "message": "Turn off the Air Conditioning" }

Response: { "reply": "Done! The AC has been turned OFF.", "steps": [...] }

The endpoint performs the following in order:

1. Decodes JWT token and extracts user_id
2. Creates or retrieves the user's ADK session (session_id = "session_{user_id}")
3. Sets current_user_id in ContextVar
4. Runs the agent loop via runner.run_async()
5. Captures tool_call and tool_result events into steps[]
6. Returns the final reply and full agent loop trace

Each user gets an isolated session so conversation history never leaks between accounts.

ChatWidget Agent Mode

The existing VoltBot ChatWidget was extended with a third Agent mode alongside AI and RAG modes. The Agent mode sends the JWT token in the Authorization header and calls the /api/v1/agent endpoint.

After the agent responds, the ChatWidget fires a browser custom event:

```
javascript
```

```
window.dispatchEvent(new CustomEvent("agent:device:updated"));
```

The Devices page listens for this event and automatically re-fetches device data from MongoDB, so the toggle switches update instantly without requiring a page refresh.

Google ADK Setup

ADK is installed via pip:

```
pip install google-adk
```

Credentials are stored in agents/.env:

```
GEMINI_API_KEY=your_key_here
```

```
GOOGLE_GENAI_USE_VERTEXAI=false
```

ADK web is launched for local testing:

```
python -m adk web agents
```

5. Output & Results

Checkpoint Demo Result

The Week 4 checkpoint required sending "Turn off the Air Conditioning" to /api/v1/agent and demonstrating that the agent calls toggle_device with the correct device and state.

API Response:

json

```
{
  "reply": "Done! The Air Conditioning has been turned OFF.",
  "steps": [
    {
      "step": "tool_call",
      "tool": "toggle_device",
      "args": { "device_name": "Air Conditioning", "state": false }
    },
    {
      "step": "tool_result",
      "tool": "toggle_device",
      "result": "Air Conditioning (Living Room) has been turned OFF successfully."
    }
  ]
}
```

Technologies Added in Week 4

Technology	Purpose
Google ADK 2.0.0	Agent Development Kit framework
gemini-2.5-flash	LLM for sub-agents tool calling
gemini-2.0-flash	LLM for router agent routing
python-jose	JWT decoding in agent endpoint
contextvars	Per-request user ID isolation

Agent Capabilities Demonstrated

- Natural language device control — "Turn off the fan", "Is the AC on?"
- Multi-device operations — "Turn on all the lights"
- Device registration — "Add a heater in the bedroom"

- Energy advice — "How much does running my AC cost per month?"
- Router switching — agent correctly routes between specialists based on intent
- Real-time UI sync — Devices page updates automatically after agent actions

Platform Output Images

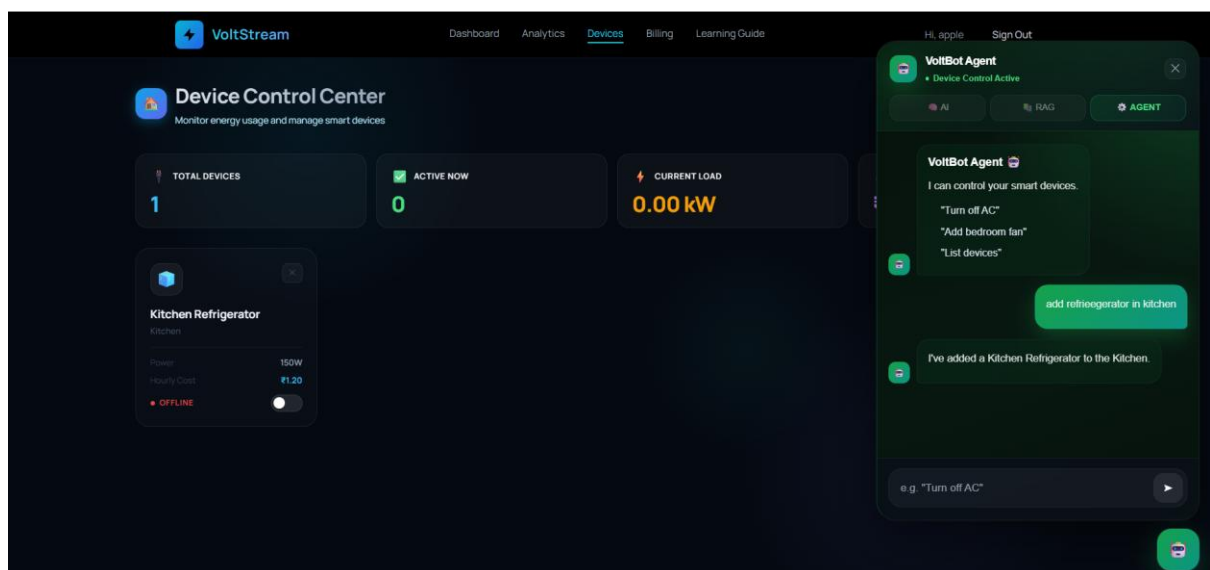
(Architecture diagram — insert here)

(ADK web Events tab showing agent loop trace)

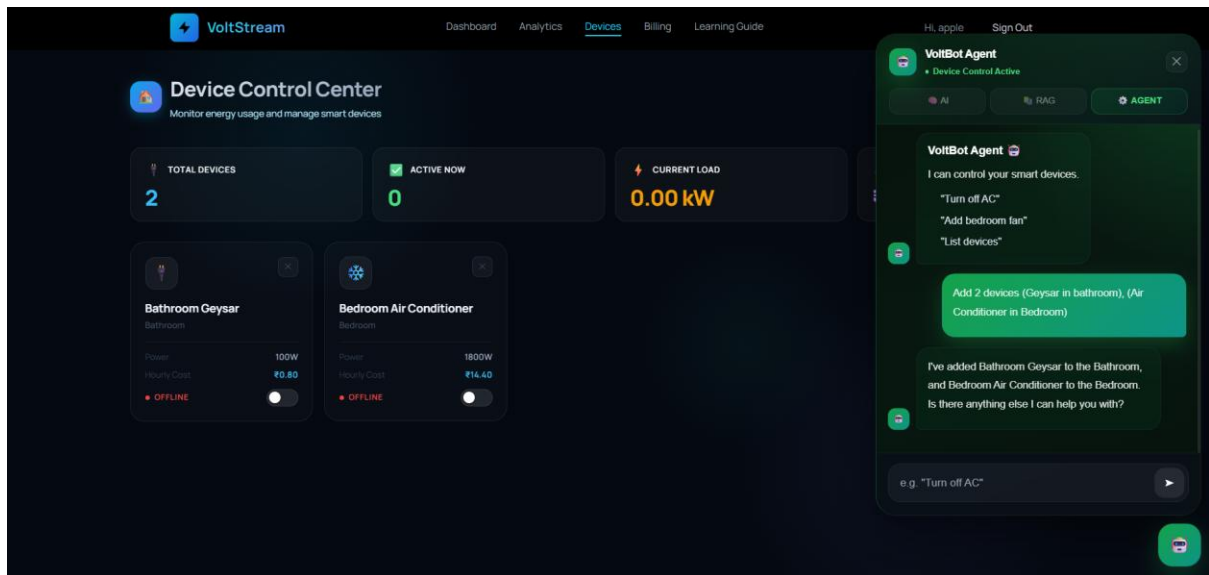
(ChatWidget Agent mode showing tool call and result)

(Devices page showing real-time toggle update)

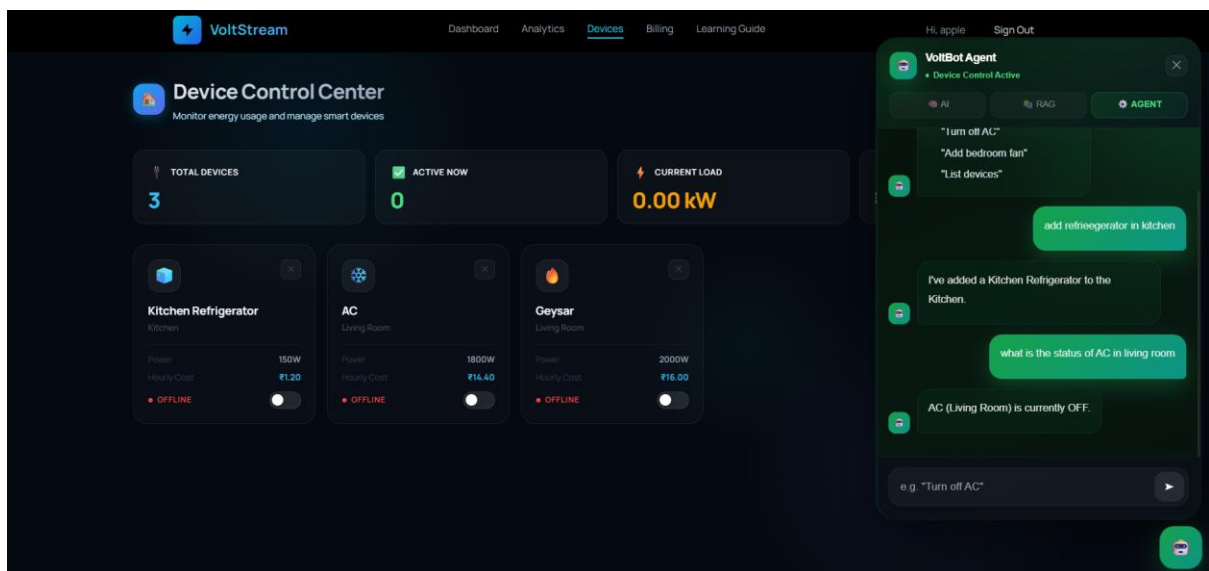
1. Adding Single Device



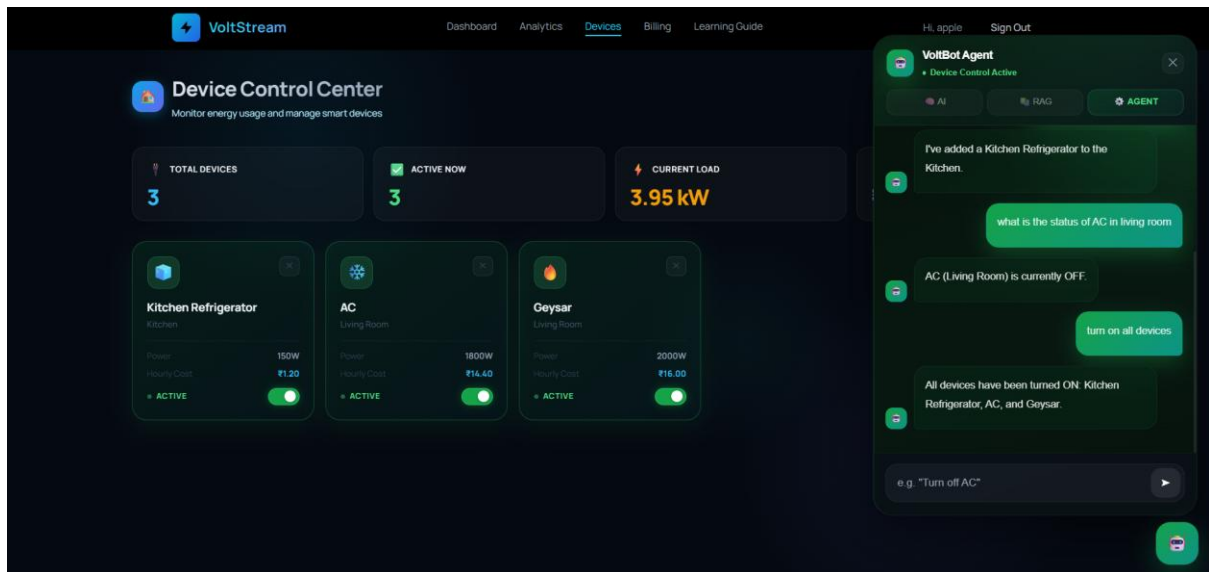
1. Adding Multiple devices



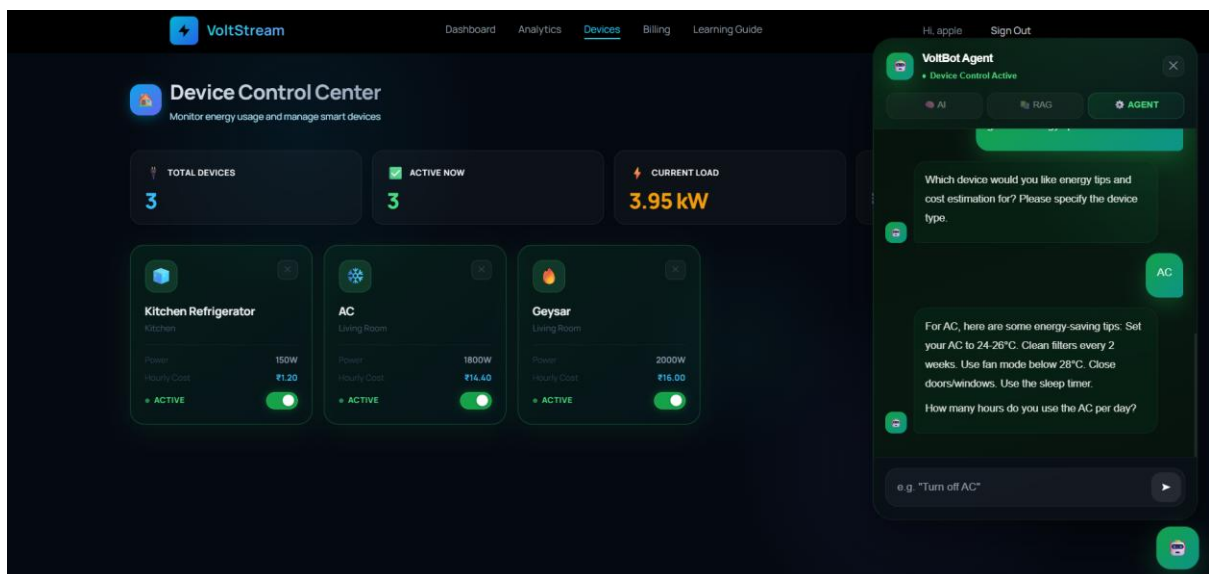
2. Checking the status of device



3. Turning on multiple devices



4. Energy Tips



5. Cost Estimation for single device

 VoltStream

[Dashboard](#) [Analytics](#) [Devices](#) [Billing](#) [Learning Guide](#)

Hi, apple [Sign Out](#)

 **Device Control Center**
Monitor energy usage and manage smart devices

TOTAL DEVICES

3

ACTIVE NOW

3

CURRENT LOAD

3.95 kW

 **Kitchen Refrigerator**
Kitchen

Power: 150W
Hourly Cost: ₹1.20
ACTIVE

 **AC**
Living Room

Power: 1800W
Hourly Cost: ₹14.40
ACTIVE

 **Geysar**
Living Room

Power: 2000W
Hourly Cost: ₹16.00
ACTIVE

VoltBot Agent
• Device Control Active

AI

RAG

AGENT

For AC, here are some energy-saving tips: Set your AC to 24-26°C. Clean filters every 2 weeks. Use fan mode below 28°C. Close doors/windows. Use the sleep timer.

How many hours do you use the AC per day?

5

Your estimated monthly cost for using the AC 5 hours a day is ₹2160.00, consuming 270.00 kWh.

e.g. "Turn off AC"